



Gestione delle Notifiche

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Cosa sono le notifiche

Una notifica è un messaggio che Android mostra all'utente al di fuori dell'interfaccia dell'app: nella barra di stato, nel drawer, sulla lock screen e come heads-up.

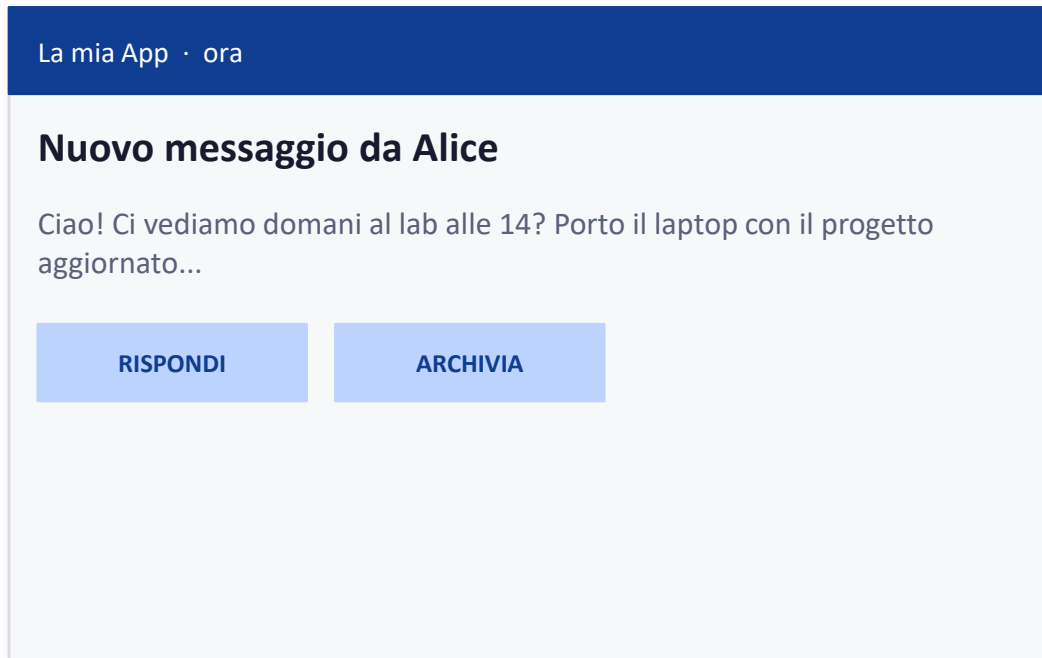
- Informano l'utente di eventi senza aprire l'app (nuovi messaggi, download completato, ecc.).
- Appaiono nella status bar come piccola icona e nel notification drawer come card espandibile.
- Possono contenere azioni: il tocco apre l'app, i pulsanti eseguono operazioni dirette.
- Da Android 8.0 (API 26) ogni notifica DEVE appartenere a un NotificationChannel.
- Da Android 13 (API 33) l'app DEVE richiedere il permesso POST_NOTIFICATIONS a runtime.
- Sono il modo principale per comunicare dal background verso l'utente.



Una notifica inutile o troppo frequente porta l'utente a disabilitare il canale o a disinstallare l'app.

Anatomia di una notifica

Ogni notifica ha componenti obbligatori e opzionali che determinano cosa vede l'utente.



① Small icon + app name + timestamp

② Titolo (contentType)

③ Testo (contentType)

④ Azioni (addAction)

⑤ Large icon (opzionale)

⑥ Stile espanso (BigTextStyle, ecc.)

NotificationChannel (API 26+)

Da Android 8.0 ogni notifica deve appartenere a un canale. L'utente può gestire ogni canale indipendentemente (suono, vibrazione, visibilità).

```
fun createNotificationChannels(context: Context) {  
    val manager = context.getSystemService(NotificationManager::class.java)  
  
    // Canale messaggi: alta importanza (heads-up)  
    val messages = NotificationChannel(  
        "messages",           // ID univoco  
        "Messaggi",           // Nome visibile all'utente  
        NotificationManager.IMPORTANCE_HIGH  
    ).apply {  
        description = "Notifiche per nuovi messaggi"  
        enableVibration(true)  
        setShowBadge(true)  
    }  
  
    // Canale aggiornamenti: bassa importanza (silenzioso)  
    val updates = NotificationChannel(  
        "updates", "Aggiornamenti", NotificationManager.IMPORTANCE_LOW  
    ).apply { description = "Aggiornamenti non urgenti" }  
  
    manager.createNotificationChannels(listOf(messages, updates))  
}
```

Kotlin

Livelli di importanza

L'importanza del canale determina come la notifica interrompe l'utente.

Importanza	Comportamento	Esempio
IMPORTANCE_HIGH	Heads-up: compare in alto sullo schermo con suono	Messaggio in arrivo, allarme
IMPORTANCE_DEFAULT	Appare nel drawer con suono	Email, reminder
IMPORTANCE_LOW	Nel drawer, senza suono	Aggiornamento social, news
IMPORTANCE_MIN	Solo nella barra di stato, nessuna interruzione	Meteo, suggerimenti
IMPORTANCE_NONE	Canale disabilitato dall'utente	—



L'utente ha il controllo finale

Dopo la creazione del canale, solo l'utente può modificarne l'importanza dalle impostazioni. L'app non può cambiarla da codice.



Scegli bene l'importanza: troppi heads-up = utente infastidito = canale disabilitato.

Quando e dove creare i canali

I canali vanno creati il prima possibile. Il posto più comune è onCreate() della classe Application.

```
class MyApp : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        createNotificationChannels(this)  
    }  
}  
  
// AndroidManifest.xml:  
// <application android:name=".MyApp" ...>
```

Kotlin

Regole importanti

- Creare un canale già esistente è un no-op (nessun effetto).
- Non puoi modificare l'importanza dopo la creazione.
- Puoi rinominare o cambiare la descrizione.
- Creare i canali in Application è idempotente e sicuro.
- Raggruppa i canali con NotificationChannelGroup.



createNotificationChannel() è sicuro da chiamare ripetutamente: se il canale esiste già, la chiamata non fa nulla.

Creare una notifica base

NotificationCompat.Builder è il costruttore retrocompatibile. Richiede un context e un channel ID.

```
fun showSimpleNotification(context: Context) {  
    val notification = NotificationCompat.Builder(context, "messages")  
        .setSmallIcon(R.drawable.ic_notification) // obbligatorio  
        .setContentTitle("Nuovo messaggio") // obbligatorio  
        .setContentText("Alice: Ci vediamo domani?") // obbligatorio  
        .setPriority(NotificationCompat.PRIORITY_HIGH) // pre-API 26  
        .setAutoCancel(true) // sparisce al tocco  
        .setColor(ContextCompat.getColor(context, R.color.primary))  
        .build()  
  
    // Inviare la notifica  
    val manager = NotificationManagerCompat.from(context)  
    if (ActivityCompat.checkSelfPermission(  
        context, Manifest.permission.POST_NOTIFICATIONS  
    ) == PackageManager.PERMISSION_GRANTED) {  
        manager.notify(1001, notification) // ID univoco  
    }  
}
```

Kotlin

PendingIntent: reagire al tocco

Un PendingIntent permette di eseguire un'azione quando l'utente tocca la notifica. È un Intent differito.

```
// Intent che apre l'Activity di dettaglio
val intent = Intent(context, DetailActivity::class.java).apply {
    putExtra("messageId", 42)
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
}

// PendingIntent wrappa l'Intent
val pendingIntent = PendingIntent.getActivity(
    context, 0, intent,
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
)

// Collegalo alla notifica
val notification = NotificationCompat.Builder(context, "messages")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Nuovo messaggio")
    .setContentText("Alice: Ci vediamo domani?")
    .setContentIntent(pendingIntent) // tocco sulla notifica
    .setAutoCancel(true)           // chiudi dopo il tocco
    .build()
```

Kotlin

Aggiungere azioni alla notifica

Le azioni appaiono come pulsanti sotto il testo della notifica (massimo 3).

```
// Azione "Rispondi" che apre l'app
val replyIntent = PendingIntent.getActivity(
    context, 1, Intent(context, ReplyActivity::class.java),
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
)

// Azione "Archivia" che avvia un BroadcastReceiver
val archiveIntent = PendingIntent.getBroadcast(
    context, 2, Intent(context, ArchiveReceiver::class.java),
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
)

val notification = NotificationCompat.Builder(context, "messages")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Nuovo messaggio")
    .setContentText("Alice: Ci vediamo domani?")
    .addAction(R.drawable.ic_reply, "Rispondi", replyIntent)
    .addAction(R.drawable.ic_archive, "Archivia", archiveIntent)
    .build()
```

Kotlin

Stile `BigTextStyle`

Quando la notifica viene espansa, mostra un testo lungo anziché troncato.

```
val style = NotificationCompat.BigTextStyle()
    .bigText(
        "Ciao! Ci vediamo domani al lab "
        + "alle 14:00? Porto il laptop con "
        + "il progetto aggiornato. Dobbiamo "
        + "finire il modulo sulle notifiche "
        + "prima della consegna di venerdì."
    )
    .setBigContentTitle("Alice")
    .setSummaryText("2 messaggi")

val notification = NotificationCompat
    .Builder(context, "messages")
    .setSmallIcon(R.drawable.ic_msg)
    .setContentTitle("Alice")
    .setContentText("Ciao! Ci vediamo...")
    .setStyle(style)
    .build()
```

Kotlin

Stili disponibili

`BigTextStyle`

Testo lungo espandibile

`BigPictureStyle`

Immagine grande

`InboxStyle`

Lista di righe (email-like)

`MessagingStyle`

Conversazione a più persone

`MediaStyle`

Controlli audio/video

`DecoratedCustomViewStyle`

Layout personalizzato con limiti

Stile MessagingStyle

Per app di messaggistica: mostra una conversazione con avatar e nomi dei partecipanti.

```
val me = Person.Builder().setName("Tu").build()
val alice = Person.Builder()
    .setName("Alice")
    .setIcon(IconCompat.createWithResource(context, R.drawable.avatar_alice))
    .build()

val style = NotificationCompat.MessagingStyle(me)
    .setConversationTitle("Progetto Mobile")
    .addMessage("Ci vediamo domani?", System.currentTimeMillis() - 60000, alice)
    .addMessage("Certo! Alle 14 al lab", System.currentTimeMillis(), me)

val notification = NotificationCompat.Builder(context, "messages")
    .setSmallIcon(R.drawable.ic_msg)
    .setStyle(style)
    .setShortcutId("alice_chat") // link alla scorciatoia conversazione
    .build()
```

Kotlin



MessagingStyle supporta le Bubbles (bolle flottanti) da Android 11.

Notifica con progress bar

Ideale per download, upload o operazioni lunghe. Si aggiorna re-inviando la stessa notification ID.

```
val NOTIF_ID = 2001
val builder = NotificationCompat.Builder(context, "downloads")
    .setSmallIcon(R.drawable.ic_download)
    .setContentTitle("Download in corso")
    .setOngoing(true)           // non swipeable
    .setOnlyAlertOnce(true)     // suono solo alla prima volta

// Mostra progresso indeterminato
builder.setProgress(0, 0, true)
manager.notify(NOTIF_ID, builder.build())

// Aggiorna con progresso determinato
for (progress in 0..100 step 10) {
    builder.setProgress(100, progress, false)
    .setContentText("$progress%")
    manager.notify(NOTIF_ID, builder.build())
    delay(500)
}

// Completato: rimuovi il progresso
builder.setContentText("Download completato")
    .setProgress(0, 0, false)
    .setOngoing(false)
manager.notify(NOTIF_ID, builder.build())
```

Kotlin

Raggruppare le notifiche

Quando si inviano più notifiche dello stesso tipo, raggrupparle per evitare di inondare il drawer.

```
val GROUP_KEY = "com.example.MESSAGES"

// Notifica singola (per ogni messaggio)
fun notifyMessage(id: Int, sender: String, text: String) {
    val n = NotificationCompat.Builder(context, "messages")
        .setSmallIcon(R.drawable.ic_msg)
        .setContentTitle(sender)
        .setContentText(text)
        .setGroup(GROUP_KEY)
        .build()
    manager.notify(id, n)
}

// Summary (la "testata" del gruppo)
val summary = NotificationCompat.Builder(context, "messages")
    .setSmallIcon(R.drawable.ic_msg)
    .setContentTitle("3 nuovi messaggi")
    .setStyle(NotificationCompat.InboxStyle()
        .addLine("Alice: Ciao!")
        .addLine("Bob: Ci vediamo?")
        .addLine("Carol: OK!"))
    .setGroup(GROUP_KEY)
    .setGroupSummary(true) // questa è la summary
    .build()
manager.notify(0, summary)
```

Kotlin

Cancellare le notifiche

Si può cancellare una singola notifica per ID, o tutte quelle della app.

```
val manager = NotificationManagerCompat.from(context)
```

Kotlin

```
// Cancella una singola notifica per ID  
manager.cancel(1001)
```

```
// Cancella TUTTE le notifiche dell'app  
manager.cancelAll()
```

```
// setAutoCancel(true) → sparisce al tocco  
// setTimeoutAfter(5000) → sparisce dopo 5s
```

Quando cancellare?

- L'utente ha letto il messaggio nell'app
- L'azione è stata completata (download)
- Il timer è scaduto (reminder)
- L'utente ha interagito con un'azione



setOngoing(true) impedisce all'utente di swipare via la notifica. Usarlo solo per operazioni attive (download, musica).

Permesso POST_NOTIFICATIONS (API 33+)

Da Android 13 l'app DEVE chiedere il permesso a runtime per mostrare notifiche. Senza il consenso, le notifiche vengono bloccate silenziosamente.

1. Manifest

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

XML



Sotto API 33

Il permesso non esiste: le notifiche sono abilitate di default.



Se l'utente nega due volte

Android non mostra più il dialog. Bisogna indirizzarlo nelle impostazioni.

2. Richiesta a runtime

```
val launcher = registerForActivityResult(  
    ActivityResultContracts  
        .RequestPermission()  
) { granted ->  
    if (granted) {  
        showNotification()  
    } else {  
        // spiega perche serve  
    }  
}  
  
// Richiedi il permesso  
if (Build.VERSION.SDK_INT >= 33) {  
    launcher.launch(  
        Manifest.permission  
            .POST_NOTIFICATIONS  
    )  
}
```

Kotlin

Firebase Cloud Messaging (FCM)

FCM permette di inviare notifiche push dal server alla app, anche quando l'app è chiusa. È il servizio standard per le push su Android.



Notification message

FCM mostra automaticamente la notifica quando l'app è in background. Nessun codice necessario.



Data message

L'app riceve i dati in `onMessageReceived()` e decide lei come mostrare la notifica. Pieno controllo.



Topic messaging

Invia a tutti gli iscritti a un topic (es. "news", "promo").
Niente gestione dei token singoli.



Token-based

Ogni dispositivo ha un token univoco. Il server lo usa per inviare a un utente specifico.

Implementare FCM: il Service

FirebaseMessagingService riceve i messaggi push. Si dichiara nel Manifest e si fa l'override di due callback.

```
class MyFirebaseService : FirebaseMessagingService() {  
  
    // Chiamato quando arriva un data message (o notification+data in foreground)  
    override fun onMessageReceived(msg: RemoteMessage) {  
        val title = msg.notification?.title ?: msg.data["title"] ?: "Notifica"  
        val body = msg.notification?.body ?: msg.data["body"] ?: ""  
        showNotification(title, body)  
    }  
  
    // Chiamato quando il token viene generato o aggiornato  
    override fun onNewToken(token: String) {  
        // Invia il token al tuo server  
        sendTokenToServer(token)  
    }  
  
    private fun showNotification(title: String, body: String) {  
        val n = NotificationCompat.Builder(this, "messages")  
            .setSmallIcon(R.drawable.ic_notification)  
            .setContentTitle(title).setContentText(body)  
            .setAutoCancel(true).build()  
        NotificationManagerCompat.from(this).notify(System.currentTimeMillis().toInt(), n)  
    }  
}
```

Kotlin

FCM: setup e token

Il servizio va dichiarato nel Manifest. Il token si ottiene anche programmaticamente.

AndroidManifest.xml

```
<service
    android:name=".MyFirebaseService"
    android:exported="false">
    <intent-filter>
        <action android:name=
            "com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

XML

Ottenere il token

```
FirebaseMessaging.getInstance().token
    .addOnSuccessListener { token ->
        Log.d("FCM", "Token: $token")
        // invia al server
        sendTokenToServer(token)
    }

// Iscrizione a un topic
FirebaseMessaging.getInstance()
    .subscribeToTopic("news")
```

Kotlin



Dipendenza Gradle

```
implementation("com.google.firebase:firebase-messaging:24.0.0") + google-services plugin + google-services.json
```

Notifiche da un'app Compose

Le notifiche non sono legate al framework UI: funzionano identicamente con View e Compose. Ecco come richiedere il permesso in Compose.

```
@Composable
fun NotificationPermissionRequest() {
    val context = LocalContext.current
    val launcher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted ->
        if (granted) showNotification(context)
    }

    Button(onClick = {
        if (Build.VERSION.SDK_INT >= 33) {
            launcher.launch(Manifest.permission.POST_NOTIFICATIONS)
        } else {
            showNotification(context) // permesso non necessario pre-API 33
        }
    }) {
        Text("Invia notifica di test")
    }
}
```

Kotlin

Best practice

DA FARE

- Crea i canali in `Application.onCreate()`
- Usa `NotificationCompat` (retrocompatibile)
- Controlla il permesso prima di `notify()`
- Raggruppa le notifiche correlate
- `setAutoCancel(true)` per notifiche temporanee
- Scegli l'importanza giusta per il canale
- Testa su API 26, 33+ e pre-26

DA EVITARE

- `IMPORTANCE_HIGH` per tutto
- Notifiche senza `PendingIntent`
- Dimenticare `FLAG_IMMUTABLE`
- Ignorare `POST_NOTIFICATIONS` su API 33+
- Notifiche troppo frequenti
- Usare `Notification.Builder` (non `Compat`)
- Canale unico per tutti i tipi

Errori comuni e soluzioni

Notifica non appare

Canale mancante (API 26+), permesso negato (API 33+), o importanza NONE.

Heads-up non compare

Il canale ha importanza < HIGH, oppure l'utente l'ha abbassata. Non possibile cambiarla da codice.

Token FCM cambia

Il token può cambiare: gestire onNewToken() e aggiornare il server.

SecurityException: FLAG_IMMUTABLE

Da API 31+ FLAG_IMMUTABLE o FLAG_MUTABLE è obbligatorio nei PendingIntent.

Notifica duplicata

Stessa ID → aggiornamento. ID diversa → nuova notifica. Controllare la logica.

Crash su notify() senza permesso

Controllare checkSelfPermission() prima di chiamare notify().

Cheat-sheet: notifiche Android

```
// 1. Creare il canale (in Application.onCreate)
NotificationChannel("messages", "Messaggi", IMPORTANCE_HIGH)
manager.createNotificationChannel(channel)

// 2. Costruire la notifica
val n = NotificationCompat.Builder(ctx, "messages")
    .setSmallIcon(R.drawable.ic_notif)
    .setContentTitle("Titolo").setContentText("Corpo")
    .setContentIntent(pendingIntent)
    .setAutoCancel(true).build()

// 3. Controllare il permesso e inviare
if (checkSelfPermission(ctx, POST_NOTIFICATIONS) == GRANTED) {
    NotificationManagerCompat.from(ctx).notify(id, n)
}

// 4. PendingIntent
PendingIntent.getActivity(ctx, 0, intent, FLAG_UPDATE_CURRENT or FLAG_IMMUTABLE)

// 5. Azioni
builder.addAction(icon, "Testo", pendingIntent)

// 6. Stili
builder.setStyle(BigTextStyle().bigText("..."))
builder.setStyle(MessagingStyle(me).addMessage("...", time, sender))

// 7. Aggiornare (stessa ID) / Cancellare
manager.notify(sameId, updatedNotification)
manager.cancel(id) // o cancelAll()
```

Kotlin

Quale tipo di notifica usare?

Un albero decisionale per scegliere lo stile e l'importanza giusti.

